Ghost Cloud: Product and Build Plan

An AI agent that ships your app, protects it, controls its costs, and tells you how to improve it.

1. What is Ghost Cloud?

Ghost Cloud is a cloud agent for developers and small teams. You connect a GitHub repo and a hosting project (Vercel first), and Ghost takes over the repetitive work around running a web app: deploying it, watching the code, tracking what it costs, and studying how real users behave on it.

Most tools do one of these jobs. Hosts deploy, linters scan, billing dashboards show costs after the fact, and analytics tools show charts. Ghost Cloud connects all of them into one loop, because it understands the codebase:

Code → Deploy → Real usage and cost → What to change next → Code

One-line pitch: Push your code, and Ghost ships it, watches it, keeps your bill under control, and tells you how to make it better.

Who it is for: solo developers, indie hackers, startups, and small teams who run web apps and APIs but have no DevOps, QA, or analytics person.

Built on: the Ghost engine (the existing Ghost CLI coding agent), wrapped in a GitHub App, secure sandboxes, and a web dashboard.

2. What it does: the four modules

Ship: deploy safely

- Detects the framework (Next.js, Vite, Express, Django, static site) and deploys through the Vercel API/CLI
- Creates a preview deploy for every pull request and posts the link in the PR
- Runs pre-deploy checks: build passes, required environment variables exist, no secrets in the diff
- Runs post-deploy health checks and compares status and speed with the previous version
- Offers one-command rollback (automatic rollback comes later)

Spend: control costs

- Tracks API usage and spend across LLM providers (OpenAI, Anthropic), hosting, and other services
- Shows a live view: "this app costs $X per day"
- Attributes cost to code: "this endpoint caused 70% of yesterday's spend"
- Sends spike alerts and suggests savings (cache this call, use a cheaper model, downgrade this plan)
- Supports budget caps: alert first, enforcement later

Guard: protect the repo

- Watches every push and PR with linters, dependency audits, secret scanning, and an AI review
- Flags risky commits (removed validation, changed auth, broke a documented API)
- Detects README and docs drift and proposes fixes as PRs
- Proposes fixes for problems it finds, never pushes directly to the main branch

Grow: improve the product

- Reads site analytics (Google Analytics first, Vercel Analytics or others where an API allows)
- Produces a weekly report with evidence-backed actions, not just charts
- Links metric changes to deploys and code: "load time on /dashboard got 1.8s slower after Tuesday's deploy, caused by an unoptimized image"
- Opens ready-to-review PRs for the top recommendations
- Tracks results afterwards: did the metric actually move?

Three levels of output for every insight: **Observe** (what happened), **Explain** (why, tied to code or a deploy), **Act** (a PR or concrete plan).

3. Why it can win

- **Code-aware cost control.** Hosts have no reason to help you spend less. Ghost can connect a bill spike to the exact line of code.
- **Closed loop.** Analytics tools stop at charts. Ghost knows the codebase, so it can act on what it sees.
- **Verified changes only.** Nothing is proposed unless the build and tests pass.
- **Sticky.** It sits in the deploy path and runs weekly, so it becomes part of the workflow.
- **Natural free funnel.** The free Ghost CLI and the cost tracker bring in users.

4. Version 1 scope

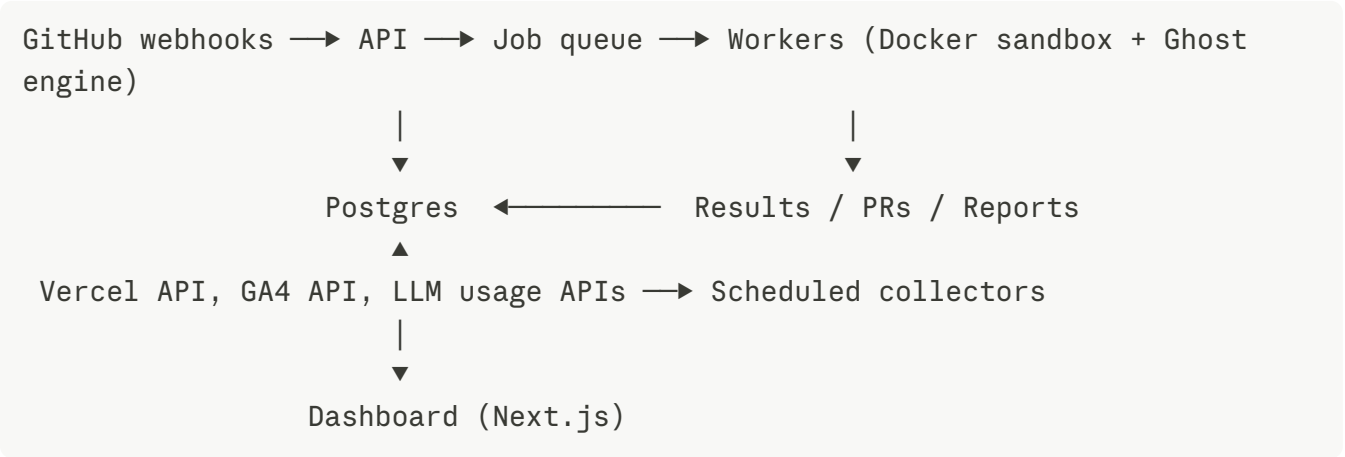
In v1: Vercel and GitHub only, JS/TS and Python projects, four thin modules, GA4 for analytics.

Not in v1: Render/Netlify/Fly, GitLab, automatic production rollback, team features, a polished dashboard, the legacy migrator product.

5. Tech stack

Layer	Choice	Why
Agent engine	Ghost CLI as a library or subprocess	Already built, so do not rewrite it
Backend API	Python + FastAPI	Same language as the engine
Database	Postgres (Supabase or Neon)	Relational data, free tiers
Job queue	Redis + RQ/Celery, or a Postgres queue to start	Scans and deploys are long-running
Sandboxes	Docker, one container per job, no network by default	Safe execution of untrusted repos
Dashboard	Next.js hosted on Vercel	Fast to build, and you dogfood your own product
GitHub	GitHub App (not OAuth)	Per-repo permissions and webhooks
LLM	Provider-agnostic wrapper	Cheap model for scans, strong model for fixes
Auth	GitHub login	Developers already have it

6. Architecture



Core modules

- **Connectors:** GitHub, Vercel, GA4, OpenAI/Anthropic usage
- **Policy engine:** decides what the agent may do without asking (permission tiers)
- **Job runner:** clone, sandbox, run Ghost, verify, produce an artifact (PR, report, or deploy)
- **Verifier:** build and tests must pass before anything is proposed
- **Cost tracker:** logs tokens and dollars for every job, including your own LLM spend

7. Data model (starter)

- `users`, `orgs`
- `projects`: repo, vercel_project_id, permission_tier, budget_cap
- `jobs`: type, status, start and finish times, llm_cost, logs_url
- `deployments`: commit, url, status, health_check_result
- `usage_events`: provider, route or endpoint, tokens, cost, timestamp
- `insights`: type, evidence (JSON), severity, status, pr_url
- `reports`: project, period, summary, created_at
- `audit_log`: every action the agent took and who approved it

8. Building each module

Ship (weeks 1 to 3)

- `ghost deploy`: detect framework, build in a sandbox, deploy via the Vercel API/CLI
- Preview deploy per PR with the URL posted as a comment
- Pre-deploy checks (build, env vars, secrets) and post-deploy health checks

- **Done when:** a push to a PR produces a preview link and a pass/fail check in under 3 minutes.

Spend (weeks 3 to 5)

- Pull usage from provider APIs (OpenAI/Anthropic usage endpoints, Vercel usage)
- Optional SDK wrapper or proxy that tags each call with its route, for code-level attribution
- Daily summary, spike alerts (email, Discord, Slack webhook), budget caps (alert-only first)
- **Done when:** it can say "yesterday cost $X; endpoint Y caused Z%".

Guard (weeks 5 to 7)

- Webhook on every push and PR: linters, dependency audit, secret scan, Ghost review
- Risky-commit flags and README/docs drift PRs
- Approval flow for anything that changes code
- **Done when:** a bad commit produces a clear warning within minutes, with a suggested fix PR.

Grow (weeks 7 to 9)

- Connect GA4 through the Data API first
- **Check in week 1 what Vercel Analytics exposes through an API.** Read access may be limited. Fallback: a small tracking snippet that Ghost injects.
- Weekly job: pull metrics, compare with previous weeks and deploy dates, detect anomalies (bounce rate, load time, funnel drop), map them to code, and generate ranked recommendations
- Every recommendation shows evidence, confidence, and effort, and pages with too little traffic are skipped
- Follow-up tracking after changes ship
- **Done when:** a weekly email contains 3 to 5 evidence-backed actions, with at least one ready PR.

9. Week-by-week schedule (12 weeks)

Week	Goal	Deliverable
1	Foundations	Repo, FastAPI skeleton, Postgres schema, GitHub App registered, Vercel API verified, analytics API check
2	Sandbox runner	Docker job runner calling Ghost, job queue, logs
3	Ship v1	`ghost deploy` and PR preview comments
4	Ship hardening and Spend start	Health checks, env var check, LLM usage collectors
5	Spend v1	Cost dashboard, daily summary, spike alerts
6	Guard v1	Webhook scans, secret and dependency checks, commit risk flags
7	Guard v2	Docs drift PRs, review comments, approval flow
8	Grow v1	GA4 connector, metric ingestion, anomaly detection
9	Grow v2	Recommendations, PR generation, weekly email report
10	Dashboard and onboarding	Connect-repo flow, project page, Stripe billing
11	Private beta	10 to 20 real users, fix what breaks
12	Public launch	Landing page, demo video, launch posts

If you fall behind, cut Guard v2 and Grow v2 first, never Ship or Spend.

10. Security and trust (non-negotiable)

- **Permission tiers:** Watch (read-only), then Preview (PRs and preview deploys), then Full (production deploys). Default is Watch.
- Production deploys need approval until the user opts in to automation
- Scoped, encrypted tokens only, never written to logs
- Sandboxes: no network (or an allowlist), CPU, memory, and time limits, destroyed after each job
- Never run repo code outside a sandbox
- Full audit log, visible to the user

- **Prompt-injection defense:** repo content, issues, and analytics data are untrusted input and must never be able to trigger deploys or change permissions
- A manual rollback is always available

11. LLM cost control

- Run static tools first (linters, audits) and call the LLM only on flagged files
- Use a cheap model for classification and summaries, a strong model only for fixes
- Cache results by commit hash
- Set a monthly LLM budget per project, tracked in `jobs.llm_cost`
- Target: average LLM cost per project under about 20% of the plan price

12. Pricing

- **Free:** 1 project, Ship and basic health, weekly cost summary
- **$19/month:** unlimited projects, Spend, Guard, alerts
- **$49/month:** adds Grow (weekly improvement plans and PRs)
- Later: free for open-source repos (marketing), and team plans with audit logs and shared budgets

13. Go-to-market

1. **Dogfood:** run Ghost Cloud on your own projects and publish the real before and after results
2. **Open-source demos:** open small, genuinely useful PRs on public repos, with maintainers in mind
3. **Content:** short videos like "Ghost found why my bill spiked" and "Ghost's weekly report improved my signup rate"
4. **Communities:** Product Hunt, Show HN, r/webdev, r/SideProject, Indian developer communities, indie hacker circles
5. **Free tier as the funnel:** the cost tracker is the hook, and bill-saving stories get shared

Treat marketing as a weekly task from week 1, not something for week 12.

14. Success metrics

- **End of week 11 (beta):** 10 or more projects connected, 5 or more using it weekly
- **30 days after launch:** 200 connected projects, 15 or more paying

- **Quality:** PR acceptance rate above 50%, few false alarms, zero security incidents
- **Unit economics:** LLM cost per project within the budget above

15. Risks and responses

Risk	Response
Vercel Analytics has no usable API	Start with GA4 or Plausible, or use a tracking snippet
Bad patches destroy trust	Verifier gate, small PRs, confidence labels
Scope is too big	Strict week-by-week cuts, Vercel and GitHub only
Hosts build similar features	Stay multi-provider and cost-aware, which hosts have little incentive to do
Security incident involving tokens	Least privilege, encryption, audit logs, minimal data retention
No distribution	Weekly marketing from day one, free tier, public demos

16. Your first 5 days

1. **Day 1:** create the GitHub App and a Vercel token, and make a "hello" API call to each
2. **Day 2:** confirm what Vercel Analytics and GA4 let you read through their APIs
3. **Day 3:** set up the Postgres schema and the FastAPI skeleton
4. **Day 4:** run Ghost inside a Docker container against a sample repo and capture its output
5. **Day 5:** wire a GitHub webhook to a queued job that posts a PR comment ("Ghost saw your push")

That last step is the first complete loop, and everything after it builds on it.

17. Product map

```
Ghost CLI       free, local coding agent (the funnel)
Ghost Cloud     Ship + Guard + Spend + Grow   ← build first
Ghost Migrator  big one-off migrations (premium add-on, later)
```